

TestPlatform

Rapport d'Execution de Campagne

Campagne #19

Projet: Projet #17

Version: N/A

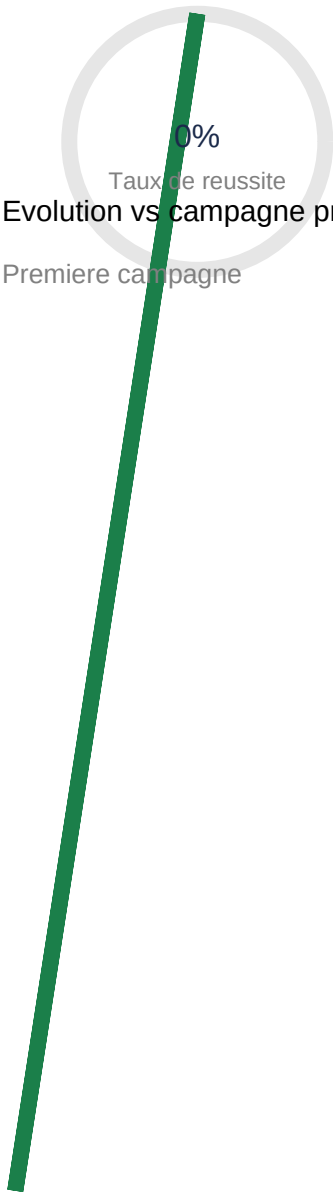
Environnement: Environnement #12

Date: 03/06/2026 10:39

ECHEC

Resume executif

Taux de reussite



Total	Succes	Echecs	Erreurs	Duree
1	0	1	0	268.4 s

Informations du contexte

Projet	Projet #17
Description	N/A

Suites

Nom	Type	Depot Git	Branche
Suite UNIT	UNIT	N/A	main

Cas de test

Titre	Type	Priorite	Risque	Script
Test #32	UNIT	N/A	N/A	IA

Environnement

URL Web	N/A
URL API	N/A
Repository	https://github.com/ilyeschaabani/Automatisation_intelligente_des_tests_applicatifs
Branche	main
Base de donnees	POSTGRESQL

Resultats details

Test #32

FAILURE

Type	UNIT	Duree	9.5 s
Script	IA	Erreur	Maven exit code : 1

Analyse IA

Test #32

Logs (extrait)	<pre>R] [ERROR] For more information about the errors and possible solutions, please read the following articles: [ERROR] [Help 1] http://cwiki.apache.org/confluence/display/MAVEN/MojoFailureException</pre>
Analyse IA	Le script de test semble sainement fonctionner sauf que la méthode <code>`testAddTestCase()`</code> n'a pas encore été écrite. Cette méthode aurait pour but d'appeler la méthode <code>`addTestCase(...)`</code> de l'objet <code>`testCaseService`</code> et vérifier si le comportement attendu est réellement atteint. Voici les points à analyser : 1. La cause probable de l'échec peut être que la méthode <code>`addTestCase(...)`</code> n'a pas encore été écrite ou qu'elle contient des instructions incomplètes ou non-fonctionnelles. 2. Pour corriger le script, il faut écrire cette méthode et lui donner les instructions qui vont bien fonctionner avec la couche de services <code>`testCaseService`</code>. 3. En ce qui concerne une suggestion d'amélioration du test, on pourrait ajouter des assertions supplémentaires pour tester le comportement en cas de succès ou d'échec de l'opération. Cependant, cela dépendrait énormément de la couche de services <code>`testCaseService`</code> et plus généralement de son comportement attendu. En résumé, le script n'est pas dans un état de fonctionnement parce que sa méthode <code>`addTestCase(...)`</code> est vide ou incomplète. Il faut donc y ajouter des instructions pour qu'elle soit correcte et fonctionne avec la couche de services <code>`testCaseService`</code>.

Evaluation UX

Aucun test UX execute pour cette campagne

Metriques

Top 5 des tests les plus lents

Test	Suite	Duree
Test #32	Suite UNIT	9.5 s

Temps d'execution par type

Type	Duree moyenne
UNIT	9.5 s

Tests instables

Aucun test instable detecte

Recommandations

- Test #32: Le script de test semble sainement fonctionner sauf que la méthode `testAddTestCase()` n'a pas encore été écrite. Cette méthode aurait pour but d'appeler la méthode `addTestCase(...)` de l'objet `testCas...`

Annexe - Scripts generes par IA

Test #32

```
package suites.unit;

import com.pfe.platform.ms_gestion.dto.request.CreateTestCaseRequest;
import com.pfe.platform.ms_gestion.dto.response.TestCaseResponse;
import com.pfe.platform.ms_gestion.entity.Project;
import com.pfe.platform.ms_gestion.entity.ProjectMember;
import com.pfe.platform.ms_gestion.entity.TestCase;
import com.pfe.platform.ms_gestion.entity.TestSuite;
import com.pfe.platform.ms_gestion.repository.ProjectMemberRepository;
import com.pfe.platform.ms_gestion.repository.TestCaseRepository;
import com.pfe.platform.ms_gestion.repository.TestSuiteRepository;
import com.pfe.platform.ms_gestion.security.SecurityUtils;
import com.pfe.platform.ms_gestion.service.LlmService;
import com.pfe.platform.ms_gestion.service.TestCaseService;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.MockedStatic;
import org.mockito.Mockito;
import org.mockito.MockitoAnnotations;
import org.testng.Assert;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Test;
import java.util.Optional;

public class TestCaseServiceTest {

    @Mock private TestCaseRepository testCaseRepository;
    @Mock private TestSuiteRepository testSuiteRepository;
    @Mock private ProjectMemberRepository projectMemberRepository;
    @Mock private LlmService llmService;
    @InjectMocks private TestCaseService testCaseService;

    private AutoCloseable mocks;
    private MockedStatic<SecurityUtils> mockedSecurity;

    @BeforeMethod
    public void setUp() {
        mocks = MockitoAnnotations.openMocks(this);
        mockedSecurity = Mockito.mockStatic(SecurityUtils.class);
        mockedSecurity.when(SecurityUtils::getCurrentUserId).thenReturn(1L);
    }

    @AfterMethod
    public void tearDown() throws Exception {
        mockedSecurity.close();
        mocks.close();
    }

    @Test
    public void testAddTestCase() {
        Long suiteId = 1L;

        Project project = new Project();
        project.setId(10L);
        project.setName("Test Project");

        ProjectMember member = new ProjectMember();
        member.setRole(ProjectMember.Role.ADMIN);
        member.setUserId(1L);
        member.setProject(project);

        TestSuite suite = new TestSuite();
```

```
suite.setId(suiteId);
suite.setType(TestSuite.TestType.UNIT);
suite.setProject(project);

CreateTestCaseRequest request = new CreateTestCaseRequest();
request.setTitle("Login test");
request.setType("UNIT");
request.setRiskLevel("LOW");
request.setPriority(1);
request.setDescriptionAI("Test the add method");

Mockito.when(testSuiteRepository.findById(suiteId))
    .thenReturn(Optional.of(suite));
Mockito.when(projectMemberRepository.findByIdAndUserId(
    Mockito.any(), Mockito.eq(1L)))
    .thenReturn(Optional.of(member));
Mockito.when(llmService.generateTestCode(
    Mockito.any(), Mockito.any(), Mockito.any(), Mockito.any()))
    .thenReturn("public class FakeTest {}");
Mockito.when(testCaseRepository.save(Mockito.any()))
    .thenAnswer(inv -> {
        TestCase tc = inv.getArgument(0);
        tc.setSuite(suite);
        return tc;
    });

TestCaseResponse response = testCaseService.add(suiteId, request);

Assert.assertNotNull(response);
Assert.assertEquals(response.getTitle(), "Login test");
Assert.assertEquals(response.getRiskLevel(), "LOW");
Mockito.verify(testCaseRepository, Mockito.times(1)).save(Mockito.any());
Mockito.verify(llmService, Mockito.times(1))
    .generateTestCode(Mockito.any(), Mockito.any(), Mockito.any(), Mockito.any());
}
```